

(Task 1 of 7) In this tutorial, we will review previously learned content.

0

(Task 2 of 7) What is the result of running this program?

Lispy | 

Lispy [Run 	Scala 3
<code>(deffun (k b)</code>	<code>def k(b : Int) =</code>
<code> (lambda (a)</code>	<code> (a : Int) =></code>
<code> (+ a b)))</code>	<code> a + b</code>
<code>(defvar foo (k 3))</code>	<code>val foo = k(3)</code>
<code>(defvar bar (k 2))</code>	<code>val bar = k(2)</code>
<code>(foo 3)</code>	<code>println(foo(3))</code>
<code>(bar 3)</code>	<code>println(bar(3))</code>

6 5

2

You predicted the output correctly 🎉🎉🎉

3

The value of `(k 3)` is a lambda function defined in an environment where `b` is bound to 3. The value of `(k 2)` is *another* lambda function defined in an environment where `b` is bound to 2. The two lambda functions are *different* values. So, the value of `(foo 3)` is 6, while the value of `(bar 3)` is 5.

Click [here](#) to run this program in the Stacker.

(Task 3 of 7) What is the result of running this program?

Lispy | 

Lispy [Run 	JavaScript
<code>(defvar n 2)</code>	<code>let n = 2;</code>
<code>(deffun (h m)</code>	<code>function h(m) {</code>
<code> (set! m 7)</code>	<code> m = 7;</code>
<code> n)</code>	<code> return n;</code>
<code>(h n)</code>	<code>}</code>
	<code>console.log(h(n));</code>

⚠️ Scala 3 behaves differently.

2

5

You predicted the output correctly 🎉🎉🎉

6

The function call binds `m` to 2. The variable assignment mutates the value of `m` to 7, but `n` is still bound to 2.

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 4 of 7) What is the result of running this program?

Lispy [Run] 

JavaScript

```
(defvar n 3)    let n = 3;
(defvar m n)    let m = n;
(set! n 6)      n = 6;
n              console.log(n);
m              console.log(m);
```

6 3

8

You predicted the output correctly 🎉🎉🎉

9

The first definition binds `n` to 3. The second definition binds `m` to the value of `n`, which is 3. The variable assignment mutates the binding of `n`, so `n` is now bound to 6. But `m` is still bound to 3.

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 5 of 7) What is the result of running this program?

Lispy [Run] 

Scala 3

```
(defvar a (mvec 88))    var a = Buffer(88)
(defvar c (mvec a 88 88)) var c = Buffer(a, 88, 88)
(set! a (mvec 76))      a = Buffer(76)
c                      println(c)
```

#(#(88) 88 88)

11

You predicted the output correctly 🎉🎉🎉

12

`a` is first bound to a one-element vector. `c` is bound to a three-element vector, of which the first element is the one-element vector. `(set! a (mvec 76))` binds `a` to a new vector. This doesn't impact `c` because the 0-th element of `c` is still the one-element vector.

Click [here](#) to run this program in the Stacker.

Lispy | 

(Task 6 of 7) What is the result of running this program?

Lispy [Run] 

Python

```

(defvar a 1)          a = 1
(deffun (foo)         def foo():
  (lambda (b)         return lambda b: a + b
    (+ a b)))         bar = foo()
(defvar bar (foo))    a = 3
(set! a 3)            print(bar(0))
(bar 0)

```

3

14

You predicted the output correctly 🎉🎉🎉

15

`a` is bound to `1`. `bar` is bound to the lambda function. `(set! a 3)` binds `a` to `3`. So, the value of `(bar 0)` is the value of `(+ 3 0)`, which is `3`.

Click [here](#) to run this program in the Stacker.

(Task 7 of 7) What is the result of running this program?

Lispy | 🎮

Lispy [Run]

Scala 3

```

(defvar t 6)  var t = 6
(deffun (f1)  def f1 =
  t)         t
(deffun (f2)  def f2 =
  (set! t 4)  t = 4
  (f1))      f1
(f2)         println(f2)
(set! t 2)   t = 2
(f1)         println(f1)

```

4 2

17

You predicted the output correctly 🎉🎉🎉

18

There is exactly one variable `t`. The `t` in `(set! t 4)` refers to that variable. Calling `f2` evaluates `(set! t 4)`, which mutates `t`. When the value of `t` is eventually printed, we see the new value.

Click [here](#) to run this program in the Stacker.

You have finished this tutorial 🎉🎉🎉

Please [print](#) the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1781422912451

